Plotting Data using Matplotlib

Line · Bar · Scatter · Customisation · Pandas Plot

NCERT · Informatics Practices · Class XII

In this Chapter

1. Introduction to Data Visualisation
2. Plotting using Matplotlib (install & import)
3. Components of a Plot
4. Line Charts with plot()
5. Customisation — Title, Labels, Grid
6. Markers, Colours, Line Styles
7. Other Plot Types (bar, scatter, hist, pie…)
8. The Pandas plot() Method
9. kind parameter & DataFrame plotting

4.1 Introduction

Data visualisation means graphical or pictorial representation of data using graphs, charts, etc.

Purpose of plotting

- Visualise variation or relationships between variables
- Make results of analysis easier to understand
- Communicate information effectively to users
- Support decision-making in health, finance, science, engineering…

4.2 Plotting using Matplotlib

Matplotlib creates static, animated and interactive 2-D plots.

```
pip install matplotlib
import matplotlib.pyplot as plt   # plt is the usual alias
```

A figure is the overall window. It contains a plotting area, legend, axis labels, ticks, title, etc.

Always give a chart title, label the axes, and provide a legend when multiple series are plotted.

Basic Line Chart with plot()

```
import matplotlib.pyplot as plt

date = ["25/12", "26/12", "27/12"]
temp = [8.5, 10.5, 6.8]

plt.plot(date, temp)    # x versus y
plt.show()              # display the figure

# Save the figure
plt.savefig('temp.png')
```

4.3 Customisation of Plots

Essential customisations for clear charts:

```
plt.xlabel("Date")
plt.ylabel("Temperature")
plt.title("Date wise Temperature")
plt.grid(True)
plt.yticks(temp)
plt.show()
```

4.3.1 Markers

A marker is a symbol that represents a data value on a line or scatter plot.

Marker	Meaning	Marker	Meaning
o	Circle	s	Square
*	Star	D	Diamond
^	Triangle up	v	Triangle down
+	Plus	x	X
.	Point	p	Pentagon

4.3.2 Colours

Pass character codes or colour names to the color parameter.

Code	Colour	Code	Colour
'b'	blue	'g'	green
'r'	red	'c'	cyan
'm'	magenta	'y'	yellow
'k'	black	'w'	white

4.3.3 Linewidth and Line Style

```
plt.plot(x, y,
         marker='*',
         markersize=10,
         color='green',
         linewidth=2,
         linestyle='dashdot')

# linestyle options: 'solid', 'dotted', 'dashed', 'dashdot'
```

Complete Customised Line Chart

```
height = [121.9, 124.5, …, 162.6]
weight = [19.7, 21.3, …, 43.2]
df = pd.DataFrame({'height': height, 'weight': weight})

plt.xlabel('Weight in kg')
plt.ylabel('Height in cm')
plt.title('Average weight with respect to average height')
plt.plot(df.weight, df.height,
         marker='*', markersize=10,
         color='green', linewidth=2, linestyle='dashdot')
plt.show()
```

Other Plot Types (pyplot)

Function	Purpose
plot()	Line / markers
bar()	Vertical bar chart
barh()	Horizontal bar chart
hist()	Histogram
pie()	Pie chart
scatter()	Scatter plot
boxplot()	Box-and-whisker plot

4.4 The Pandas plot() Function

Series and DataFrame have their own .plot() method (wrapper around pyplot).

```
s.plot()           # Series
df.plot()          # DataFrame
df.plot(kind='…') # choose plot type
```

You can still use plt.title(), plt.xlabel(), etc. after calling df.plot().

kind Parameter — Plot Types

kind =	Plot type
'line'	Line plot (default)
'bar'	Vertical bar plot
'barh'	Horizontal bar plot
'hist'	Histogram
'box'	Boxplot
'area'	Area plot
'pie'	Pie plot
'scatter'	Scatter plot

Pandas Line Chart Example (Mela Sales)

```
df = pd.read_csv("MelaSales.csv")

df.plot(kind='line',
        color=['red','blue','brown'])

plt.title('Mela Sales Report')
plt.xlabel('Days')
plt.ylabel('Sales in Rs')
plt.show()

# Legend is shown automatically
```

Customising a Pandas Line Plot

```
df.plot(kind='line',
        color=['red','blue','brown'],
        marker='*',
        markersize=10,
        linewidth=3,
        linestyle='--')

# Optional: change x-ticks
plt.xticks(ticks, labels)
```

Choosing the Right Chart

Line	Trends / change over time (continuous data)
Bar / Barh	Compare categories or discrete groups
Scatter	Relationship between two numeric variables
Hist	Distribution of a single numeric variable
Pie	Parts of a whole (percentages)
Box	Spread, median and outliers

Customisation Functions Summary

Function	Purpose
title()	Set chart title
xlabel() / ylabel()	Set axis labels
xticks() / yticks()	Set tick locations & labels
grid()	Show / hide grid lines
legend()	Place a legend
savefig()	Save figure to file
show()	Display the figure

Key Takeaways

1. Matplotlib (pyplot) is the foundation for plotting in Python.
2. Always label axes and give a meaningful title.
3. Use markers, colours and line styles to make charts clearer.
4. Pandas DataFrame/Series have a convenient .plot() method.
5. Choose the plot type (kind) according to the nature of the data.
6. savefig() lets you store the figure for reports or presentations.

Chapter Summary

1. Visualisation turns numbers into insights that are easy to grasp.
2. plt.plot(x, y) creates a basic line chart; show() displays it.
3. Customise with title, xlabel, ylabel, grid, markers, colour, linestyle.
4. Many chart types: line, bar, scatter, hist, pie, box…
5. df.plot(kind=…) plots DataFrame columns directly.
6. Combine Pandas plotting with pyplot functions for full control.

Thank You

Practice the Programs from the chapter.
Try different markers, colours and kind values.